Web Scraping — Internship Assignment

AI-Powered Job Matching Platform

Role	Scraper / Data Collection Intern
Duration	5–7 Days
Stack	Python · requests / Selenium · BeautifulSoup · JSON

About the Project

We are building a job matching platform that helps Indian job seekers find the best openings across major companies — without them manually browsing dozens of career portals. At the heart of this platform is a **database of live job listings** scraped directly from each company's career site. This keeps our data fresh, accurate, and free from the noise of aggregator platforms like Naukri or LinkedIn.

Each job listing is stored in a **normalised format** with the following fields:

Field	Type	Description
title	str	Job title as listed on the career page
company	str	Company name
location	str	City / region (e.g. "Bangalore", "Pan India")
work_mode	str	"remote" | "hybrid" | "on-site"
seniority	str	"intern" | "junior" | "mid" | "senior"
business_unit	str	Division / team (e.g. "Engineering", "Data Science")
required_skills	list[str]	Skills extracted from the JD (e.g. ["Python", "SQL"])
preferred_skills	list[str]	Good-to-have skills mentioned in the JD
description	str	Full job description text
apply_url	str	Direct link to the application page
scraped_at	str	ISO timestamp of when this listing was collected

Types of Career Sites We Scrape

Different companies host their jobs on different platforms. Your scraper must be able to handle at least **two of the three types** listed below:

Platform Type	How it works	Example Companies

Workday ATS	Clean JSON API — easiest to scrape. No JavaScript rendering needed.	Accenture, Google India, Novartis, Syngenta, Fidelity
SmartRecruiters ATS	REST API with pagination. Returns JSON with job listings.	Sanofi, Wipro Digital, some mid-size firms
Custom Career Portal (Selenium)	JavaScript-heavy page that needs a browser to render job cards.	TCS iBegin, Infosys BPM, Cognizant Careers

Your Task

Build scrapers for **exactly 2 companies** from the list below — choose one that uses an API (Workday or SmartRecruiters) and one that requires Selenium. Collect at least 20 live listings per company and output them in the normalised JSON format above.

Company	Platform	Starting URL
Accenture India	Workday API	accenture.wd3.myworkdayjobs.com/en-US/Accenture_Careers
Infosys BPM	Selenium (custom)	career.infosys.com
TCS	Selenium (iBegin)	ibegin.tcs.com/jobs
Sanofi India	SmartRecruiters	careers.sanofi.com (filter: India)
Wipro	Workday API	wipro.wd3.myworkdayjobs.com/en-US/wipro
Cognizant	Selenium (custom)	careers.cognizant.com/global/en

Task 1: Scrape Company A (API-based)

Pick Accenture, Wipro, or Sanofi. Inspect the network requests on their careers page (use browser DevTools → Network tab) to find the underlying API endpoint. Write a Python script using the requests library to paginate through all results and collect at least 20 listings.

Task 2: Scrape Company B (Selenium-based)

Pick TCS, Infosys, or Cognizant. Use Selenium to load the careers page in a headless browser, wait for job cards to render, then extract the data. Handle pagination to collect at least 20 listings.

Task 3: Normalise the Data

For both scrapers, map whatever fields the site returns into the standard schema above. Parse required_skills and preferred_skills as Python lists (not a single string). Add a scraped_at field with the current ISO timestamp.

Task 4: Save Output

Write the combined results from both scrapers to a single file: **jobs_output.json** — a JSON array of normalised job objects. Also save a brief **run_log.txt** showing how many jobs were collected per company and any errors encountered.

Expected Output — jobs_output.json (example entry)

Each entry in your output file should look like this. The structure must be consistent across both companies you scrape:

```
[
  {
    "title": "Associate Software Engineer",
    "company": "Accenture",
    "location": "Bangalore",
    "work_mode": "hybrid",
    "seniority": "junior",
    "business_unit": "Technology",
    "required_skills": ["Java", "Spring Boot", "REST APIs", "SQL", "Git"],
    "preferred_skills": ["Microservices", "Docker", "Agile"],
    "description": "Join Accenture's Technology practice to build enterprise
                    applications using Java and Spring Boot. You will work with
                    cross-functional teams across client sites in India...",
    "apply_url": "https://accenture.wd3.myworkdayjobs.com/en-US/Accenture_Careers/job/...",
    "scraped_at": "2025-06-12T09:45:32+05:30"
  },
  {
    "title": "IT Analyst",
    "company": "TCS",
    "location": "Pune",
    "work_mode": "on-site",
    "seniority": "mid",
    "business_unit": "Digital Transformation",
    "required_skills": ["Python", "SQL", "Power BI", "ETL"],
    "preferred_skills": ["Azure", "Databricks"],
    "description": "TCS Digital Transformation team is hiring IT Analysts with
                    strong Python and data skills to support client analytics
                    migration projects...",
    "apply_url": "https://ibegin.tcs.com/jobs/view?jobId=...",
    "scraped_at": "2025-06-12T09:52:11+05:30"
  }
]
```

Expected run_log.txt:

```
[2025-06-12 09:45:00] Starting Accenture scraper (Workday API)...
[2025-06-12 09:45:08] Page 1: 20 jobs fetched
[2025-06-12 09:45:11] Page 2: 20 jobs fetched
[2025-06-12 09:45:13] Page 3: 14 jobs fetched — end of results
[2025-06-12 09:45:13] Accenture: 54 jobs collected
```

```
[2025-06-12 09:46:00] Starting TCS scraper (Selenium headless)...
[2025-06-12 09:46:22] Page 1 loaded — 25 job cards found
[2025-06-12 09:47:01] Page 2 loaded — 25 job cards found
[2025-06-12 09:47:40] Page 3 loaded — 18 job cards found — no next page
[2025-06-12 09:47:40] TCS: 68 jobs collected

[2025-06-12 09:47:41] Total jobs saved to jobs_output.json: 122
[2025-06-12 09:47:41] Done.
```

Evaluation Criteria

Criterion	What we look for	Weight
Schema Compliance	All required fields present and correctly typed	25%
Data Quality	Skills parsed as lists, descriptions are full text (not snippets)	25%
Both Scraper Types	One API-based + one Selenium-based scraper working	20%
Error Handling	Gracefully handles network errors, missing fields, pagination edge cases	15%
Code Cleanliness	Readable functions, sensible variable names, comments	10%
Bonus	A 3rd company scraper, or deduplication logic, or an automated retry	5%

Hints & Tips

• **Finding the API endpoint:** Open the company careers page in Chrome → press F12 → go to the *Network* tab → filter by "Fetch/XHR" → scroll through jobs on the page → look for requests returning JSON with job data. Copy that URL as your base endpoint.

• **Workday API pattern:** Most Workday URLs follow this format: *https://company.wd3.myworkdayjobs.com/wday/cxs/company/CareerPage/jobs* — try a POST request with body *{"limit": 20, "offset": 0}* to get started.

• **Selenium tip:** Always use *WebDriverWait* with expected conditions instead of *time.sleep()*. This makes your scraper faster and more reliable.

• **Skill extraction:** Job descriptions rarely have a clean "skills" field. Use keyword matching against a known skills list (Python, SQL, Java, etc.) to populate required_skills.

• **Respect rate limits:** Add a small delay (0.5–1 second) between requests. Never hammer a server with rapid-fire calls.

■■ *Important: Only scrape publicly accessible job listings — no login required, no paywalled content. This is standard practice and perfectly legal for publicly listed jobs.*

What to Submit

1. Your Python scraper files (one per company, clearly named e.g. **accenture_scraper.py**, **tcs_scraper.py**).
2. A **jobs_output.json** file — the actual scraped output (minimum 20 entries per company).
3. A **run_log.txt** from your most recent run.
4. A brief **README.md** explaining how to install dependencies and run each scraper.
5. Push everything to a **public GitHub repo** and share the link.

Questions? Reply to your Internshala message thread.

We review submissions on a rolling basis — quality over speed!